

07 · Tracing und Timing-Analyse

Beobachten, ohne die Messung mit der Ursache zu verwechseln · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 40

Willkommen in der Systemanalyse!

- Siebter Termin: eine Messfrage stellen, eine Quelle wählen, die Grenze nennen
- Zielchip wie in VL6: STM32H755, M7 Bare-Metal, M4 FreeRTOS
- Ein Trace ist Evidenz für **beobachtete** Läufe
- Er ist kein WCET-Beweis und kein Normenstempel
- Im Repo liegen kein winIDEA-Projekt und kein BlueBox-Modell

Notizen

1 / 40

Heisenbug hier: der Fehler ändert sich, weil die Beobachtung Code, Layout oder Zeit verändert. Das ist keine Quantenmechanik. Priority Inversion und Race sind verschiedene Mechanismen aus VL4.

Folie

2 / 40

Agenda für heute

1. Einfluss der Beobachtung
2. CoreSight: ITM, DWT, ETM
3. Was eine Zahl bedeutet
4. Werkzeugkette und Zeitbasis
5. OS-Sicht und Labor, mit offenen Hardwarepunkten

Notizen

2 / 40

UART-Rechnung und Abbildung nutzen denselben Text „Hallo“, fünf Byte, etwa 434 µs Übertragung, nicht CPU-Zeit.

Recap: Unser Multicore-System

- VL6: Ownership, Barriere, non-cacheable Region
- Heute beobachten wir denselben Vertrag
- DWT sieht konfigurierte **CPU-Zugriffe** des gewählten Kerns
- DWT beweist nicht, was im SRAM hinter dem M7-Cache steht
- Ein Dual-Core-Debug beweist weder parallelen Trace noch SWO

Notizen

3 / 40

Drei Sichten getrennt halten: M7-Store, M4-Lesewert, SRAM auf einem benannten Zugriffsweg.

Folie

4 / 40

Der Beobachter-Effekt in der Physik

Analogie, kein physikalisches Gesetz für jede Messung:

- extra Befehle, Logs, anderes Scheduling
- anderes Layout, anderer Cachezustand
- Halt, Buszugriffe des Debuggers, voller Tracepuffer

GPIO, Zykluszähler, ITM, DWT und ETM beantworten verschiedene Fragen und stören verschieden stark.

Notizen

4 / 40

„Non-intrusive“ heißt: Hardwaretrace kann ohne zusätzliche Anwendungsbefehle und ohne Halt aufzeichnen. Aktivierung, Puffer, Überlauf und Stall bleiben zu prüfen. ITM bleibt Software.

Klassisches Debugging: Die `printf`-Gefahr

„Hallo“, 8N1: 5 Byte = 50 Bit.

$$50/115200 \approx 434 \mu\text{s}$$

Das ist die **Übertragungsdauer**, nicht automatisch CPU-Zeit und nicht der Halt des Chips.

Weg	CPU während der Bits
Polling	bleibt in der Ausgabe
IRQ	kurz, UART läuft weiter
DMA	Start kurz, Transfer ohne CPU

Formatierung, Locks und Puffer kommen dazu. `printf` ist auf dem zeitkritischen Messpfad ungeeignet. Außerhalb bleibt begrenztes Logging möglich.

Ein verpasster Termin ist ein Deadline Miss, kein automatischer Crash.

Notizen

5 / 40

Didaktische Rechnung: 10 μs Arbeit plus 434 μs blockierende Übertragung gegen eine angenommene Frist von 50 μs . Dieselbe Zeichenkette in der Zeichnung, nicht „Hallo Welt“. Zehn Byte wären etwa 868 μs und sind hier nicht das Beispiel.

Folie

6 / 40

Visualisierung: Der Beobachter-Effekt

Lehrstreifen, keine Messung:

- nur Rechnung: 10 μs , Frist 50 μs , gehalten
- dieselbe Rechnung plus blockierendes „Hallo“: 444 μs , Frist gerissen

Der GPIO-Puls von HIGH bis LOW enthält jede Präemption dazwischen. Ein direkter BSRR-Schreibzugriff ist nicht „1–2 Takte bis zur sichtbaren Flanke“: Bus, HAL und Pad bleiben außen vor.

Notizen

6 / 40

Frage: Welche Zeit misst der Puls? Die verstrichene Zeit zwischen den beiden Stores, inklusive Unterbrechungen, nicht die reine Funktions-CPU-Zeit.

Klassisches Debugging: Breakpoints

- Der **betroffene** Kern hält an
- Der andere Kern, DMA und Peripherie können weiterlaufen
- DBGMCU kann einzelne Timer oder Watchdogs einfrieren, wenn das so konfiguriert ist
- Der NVIC ist keine unbegrenzte FIFO: Pending kann Flanken zusammenfassen
- Puffer können überlaufen
- „50 Interrupts, dann sicher Stack Overflow“ streichen

Nach Resume ist nur rekonstruierbar, was noch in Registern, FIFOs oder Pending steht.

Notizen

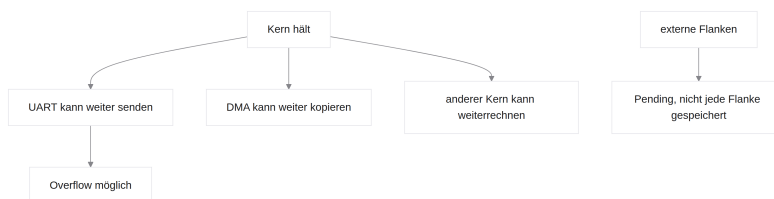
7 / 40

Frage: Welche Ereignisse sind nach dem Weiterlaufen noch da? Nicht jede verlorene Flanke.

Folie

8 / 40

Das Problem des “Halt-States”



Eine zweite Spur: Timer eingefroren, **wenn** das Freeze-Bit gesetzt ist.

Notizen

8 / 40

Halt-Debugging, laufende DAP-Lesezugriffe, Softwarelogs und Hardwaretrace sind vier verschiedene Eingriffe. SWD heißt nicht, dass jeder Speicherzugriff den Kern anhält. DAP-Verkehr ist trotzdem Buslast und kein lückenloser Trace.

Der Heisenbug

- Der Fehler verschwindet, weil die Beobachtung die Zeit oder das Layout ändert
- Race: verschränkte Zugriffe ohne Vertrag
- Priority Inversion: ein Lock plus Verdrängung des Owners
- Beides kann heikel zu beobachten sein, beides ist nicht dasselbe

Notizen

9 / 40

Die alte Notiz, die Inversion und Race in einen Satz zog, gilt nicht mehr.

Folie

10 / 40

Der Workaround: Software Instrumentierung

- GPIO und Zähler beantworten „wann bin ich hier?“
- ITM schreibt Stimulus-Wörter, es formatiert keinen Text
- `ITM_SendChar` kann auf freie Kapazität **warten**
- Voller Puffer ist keine „winzige Seltenheit“
- Laborvariante, wenn wir sie nutzen: nicht blockieren, Drops zählen, Statusbits der vorhandenen CMSIS-Version prüfen

Notizen

10 / 40

Keine Behauptung, die Kurshardware habe SWO schon nachgewiesen.

Folie

11 / 40

Wir brauchen: Non-Intrusive Observation

Präzise Formulierung:

- ohne extra Anwendungsbefehle
- ohne den Kern anzuhalten
- nicht: Einfluss immer exakt null

Prüfen: Trace an, Pfad, Puffer, Überlauf, Stall.

Notizen

11 / 40

ITM fällt aus dieser Definition heraus, weil die Anwendung schreibt.

Zusammenfassung Block 1

- 434 μ s sind Leitungszeit von „Hallo“
- Halt stoppt nicht den ganzen Chip
- GPIO misst ein Intervall, keine reine CPU-Zeit
- Hardwaretrace ist eine Quelle mit eigenen Verlusten

Notizen

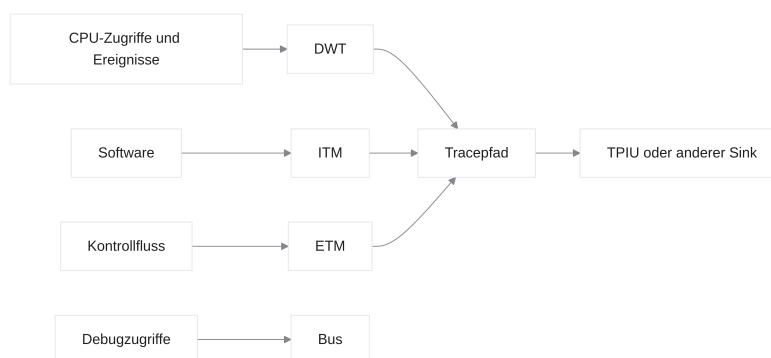
12 / 40

Nächster Block: wer die Pakete erzeugt.

Folie

13 / 40

Hardware-Trace: ARM CoreSight



Nicht drei gleiche Spione am SRAM. DWT sitzt an der konfigurierten Quelle **des Kerns**.

Was fehlt: DMA-Schreiben, der andere Kern, Cachezeilen hinter dem Store, alles außerhalb des Filters.

Notizen

13 / 40

H755-Integration und optionale Blöcke im RM0399 der Projekt-Revision prüfen. Nicht aus einem generischen M7-Bild abzeichnen. Ob beide Kerne gleichzeitig tracen, ist offen.

JTAG vs. SWD vs. Trace

Weg	transportiert	beweist nicht
SWD/JTAG	Debug	lückenlosen Trace
SWO	oft ITM/DWT	volles ETM
parallele Trace-Pins	hohe Rate, wenn vorhanden	dass das Nucleo sie hat
interner Puffer	kurz, on-chip	lange Historie

UM2408: STDC14/MIP10 am NUCLEO-H755ZI-Q ist kein Nachweis eines 20-poligen parallelen Trace-Steckers. Pinout, Raster und Adapter am Schaltplan prüfen. Pinmux, ST-LINK-Trennung, GND, VTref, Pegel, Reset.

Notizen

14 / 40

Ein 20-poliger Debugstecker und ein Cortex-Trace-Stecker sind nicht dasselbe. Keine Buchse zeichnen, die das Board nicht hat.

Folie

15 / 40

Modul 1: Das ITM (Instrumentation Trace)

- Stimulus-Port
- Software muss schreiben
- Warten oder Verwerfen ist eine Implementierungsentscheidung
- Drops dokumentieren, sonst sieht die Lücke wie „nichts passiert“ aus

Notizen

15 / 40

CMSIS-Version im Cube-Paket lesen, bevor ein Statusbit benannt wird.

Modul 2: Das DWT (Data Watchpoint & Trace)

- Wenige Komparatoren, Konflikt mit Watchpoints
- Adresse, Breite, Richtung, Maske, Datenwert und Zeitstempel hängen von der Implementierung ab
- Sie sehen nicht automatisch den anderen Kern oder DMA
- Ein M7-Store „1“ kann im Trace stehen, während Write-Back den Wert dem M4 vorenthält
- Eine flache Kurve beweist nicht „SRAM blieb 0“

Fehlende Pakete können auch heißen: falscher Kern, falsche Adresse, zu spät aufgenommen, Filter, Verlust, Kapazität.

Notizen

16 / 40

Offset aus der gemeinsamen Struktur mit `offsetof` und der Map, nicht aus einem alten Symbolnamen raten. Debugger-Memory ist ein weiterer Zugriffsweg und kann Cachewartung auslösen.

Folie

17 / 40

Modul 3: Das ETM (Embedded Trace)

- Komprimierter Kontrollfluss, plus Sync, Ausnahme, manchmal Zeit
- Nicht „ein Bit pro Sprung“
- 400 MHz sind 400 Millionen **Takte**, nicht Instruktionen
- Hypothetisches Modell: $400 \times 10^6 \text{ Ereignisse} \times 4 \text{ Byte} = 1,6 \text{ GB/s}$, nicht die echte ETM-Rate und nicht Gbit/s

Rekonstruktion braucht das passende Maschinenbild, die Protokollversion und einen vollständigen Abschnitt. Eine Lücke ist ein unbekannter Abschnitt, keine erfundene Gerade.

Notizen

17 / 40

Instruction Trace ist nicht automatisch ein Zeitstempel je Befehl. Cycle-Accurate und Timestamp extra prüfen. Keine zyklusgenaue C-Pfad-Rechnung inklusive beliebiger Stalls versprechen.

ETM komprimiert den Kontrollfluss

Schema, kein echtes Paketformat:

C-Zweig → mehrere Thumb-Befehle → komprimierte Traceereignisse → Decoder mit ELF.

Eine C-Zeile kann mehreren Befehlen entsprechen. Ohne das ELF des **denselben** Builds rekonstruiert das Tool den Pfad nicht.

Sync-Verlust: Strecke grau lassen.

Notizen

18 / 40

- g liefert Debuginfo, kein Eins-zu-eins zur Zeile. Optimierung, Inlining und entfernte Variablen bleiben.
- g und –O2 gehen zusammen. Den optimierten Lauf analysieren, nicht einen anderen Debug-Build.
- Beide Kerne: ihre eigene ELF.

Folie

19 / 40

Die TPIU und der physikalische Stecker

- TPIU ist ein möglicher Ausgang, nicht der Beweis, dass jede Spur dort ankommt
- Laboraufbau erst zeichnen, wenn Adapter, Active Probe und BlueBox-Modell feststehen
- iC5700: Ethernet-Angabe nicht als Gigabit übernehmen, solange die Gerätebeschreibung der **vorliegenden** Hardwareversion das nicht sagt
- FNet, Host-Ethernet und Target-Trace sind drei Kabelklassen
- Einschalten: BlueBox vor Target. Ausschalten: Target vor BlueBox. Stecken im ausgeschalteten Zustand. Herstelleranleitung der vorhandenen Box prüfen
- Keine Preisvergleiche

Notizen

19 / 40

Externe Digitaleingänge nur zeichnen, wenn das Zubehör da ist. Es ist nicht da im Repo.

Zusammenfassung CoreSight

Quelle	erfasst	beweist nicht
ITM	geschriebene Stimuli	rückwirkungsfreie App
DWT	gefilterte CPU-Ereignisse	SRAM hinter dem Cache
ETM	Kontrollfluss, soweit komplett	jede C-Zeile in μs
DAP	momentane Sicht	Historie

Notizen

20 / 40

Frage: Warum beweist ein M7-Store keine M4-Sichtbarkeit? Weil Cache, MPU und das Protokoll dazwischen liegen.

Folie

21 / 40

Die Pflicht zur Systemanalyse

Normen motivieren Nachweise. Sie sind hier keine Rechtsauskunft.

- ISO 26262: Straßenfahrzeuge, nicht „jede Software ist illegal“
- DO-178C: Luftfahrt-Software, Einordnung über DO-330 und AC 20-115D
- IEC 61508: funktionale Sicherheit, nicht pauschal der Medizinstandard

Trace ist keines dieser Dokumente als einziges Pflichtwerkzeug. Aussagen nur mit Ausgabe und Abschnitt, wenn wir sie wirklich zitieren. Das tun wir hier nicht im Detail.

Notizen

21 / 40

Kein „ISO-konform beweisen“ aus einem Tracefenster.

Folie

22 / 40

Metrik 1: Empirische WCET (Mess-WCET)

$\max(C_{\text{beobachtet}})$ ist das Maximum **dieser** Läufe.

Es ist keine obere Schranke für alle zulässigen Ausführungen. Ein ungetesteter Pfad kann länger sein.

Der RM-Test bleibt der aus VL2: hinreichend $n(2^{1/n} - 1)$, Grenzwert $\ln 2 \approx 69,3\%$ für große n . Ein Messmaximum ohne konservative Absicherung ist kein C_i für diesen Beweis.

Eine Million Tests zeichnen kein garantiertes Ende der Verteilung. Qualität: Sync, passende ELF, keine Verluste, bekannte Zeitbasis.

Metrik 2: Deadline-Verifizierung

Job: Freigabe r , Start s , Fertig f , Deadline $d = r + D$.

$$R = f - r, \quad \text{Slack} = d - f$$

Ein negativer Slack verletzt **diese** Frist. Daraus folgt kein „illegal im Straßenverkehr“. OS-Awareness kennt Freigabe und Deadline nicht von selbst. Die Ereignisse müssen im Trace oder im Protokoll liegen.

Frage: Woher kennt das Tool die Deadline? Nur wenn wir sie konfigurieren oder aus Ereignissen rekonstruieren.

Metrik 3: Interrupt-Latenz

Kette, nicht eine Zahl:

1. Flanke → ISR-Eintritt
2. ISR-Laufzeit
3. Veröffentlichung → Task Ready
4. Ready → Task-Start
5. Start → sichtbare Reaktion

Folie zur „ersten Bottom-Half-Zeile“ meint eine **Dispatchlatenz** bis zu einem definierten Maschinenpunkt, nicht die IRQ-Eintrittslatenz und nicht die End-to-End-Reaktion.

Eine Messung ist keine obere Schranke. Entprellung, Flanke, Messpunkt und gemeinsame Zeitbasis nennen. „Erste C-Zeile“ ist bei $-O2$ unscharf.

Welche Intervalle das vorhandene Zubehör wirklich sieht, bleibt offen, bis der Adapter feststeht.

Netto-Rechenzeit vs. Brutto-Antwortzeit

Lehrbeispiel, Overhead in diesem Bild 0:

Marke	Zeit
r	1 ms
Running	2–4 ms
Präemption	4–6 ms
Running	6–7 ms
f	7 ms
d	8 ms

CPU-Zeit des Jobs: 3 ms. $R = 6$ ms. Slack: 1 ms.

Das ist nicht die WCET.

Notizen

25 / 40

Task A, Task B und „ISR/System“ auf getrennten Zeilen denken. Nach f kann der Thread weiterlaufen oder wieder warten. Eine Jobdauer ist die CPU-Zeit dieses Jobs.

Folie

26 / 40

Metrik 4: Context Switch Overhead

PendSV ist ein Teil. Dazu können Exception-Eintritt, Rückkehr, FPU, Scheduler und die Präemption selbst gehören.

Dieselbe Zeit nicht zweimal in die Antwortzeitanalyse zählen.

2,5 μ s ist ein Beispiel, keine Kennzahl des H755.

Notizen

26 / 40

Definieren, welche Kanten der Trace als Anfang und Ende nimmt.

Ausblick: Code Coverage (Testabdeckung)

`if (a && b)`, Kurzschluss: bei `a == false` wird `b` nicht ausgewertet.

Läufe	Decision	Hinweis
TT	wahr	beide Bedingungen wahr
TF	falsch	b kippt das Ergebnis
FT	falsch	a kippt, b läuft nicht

Unabhängigkeit im vereinfachten Modell: für `a` das Paar TT/FT, für `b` das Paar TT/TF.

Statement Coverage zählt ausführbare Anweisungen, nicht Textzeilen. Decision ist nicht Condition. MC/DC braucht diese Paare und definierte Regeln bei Kurzschluss.

Instruction Trace plus Mapping ist kein automatischer MC/DC-Nachweis. Optimierung verändert das Bild. Coverage zeigt nicht, dass die Funktion richtig ist oder die Frist hält.

Die NASA-Notiz ist ein Tutorial im Kontext von DO-178B. Das ist keine „absolute Pflicht“ für dieses Labor.

Notizen

27 / 40

Kein Toolqualifikationsstempel aus dieser Folie.

Folie

28 / 40

Das Werkzeug: In-Circuit Emulator (ICE)

- BlueBox und winIDEA nur in der Version, die im Labor steht
- Modell, Active Probe, Adapter und Zusatzmodule aufschreiben, bevor eine Folie sie zeichnet
- Hier: Liste offen

Notizen

28 / 40

Herstellerseiten widersprechen sich bei Schnittstellen. Die Hardwareversion gewinnt.

Vorstellung: TASKING BlueBox

Offen, bis das Gerät da ist:

- genaue Typbezeichnung
- ob paralleler Trace, SWO oder nur Debug
- Ethernet der Box, nicht raten
- keine Gigabit-Annahme
- keine externen ADIO-Pins ohne Modul

Notizen

29 / 40

FBridge nicht mit dem Host-Ethernet-Kabel verwechseln, auch wenn die Stecker ähnlich aussehen.

Folie

30 / 40

Die IDE: winIDEA

Menüpfade der **installierten** Version. „Plugins → FreeRTOS“ nicht als ewig gültig abschreiben. Geladen wird die ELF des gemessenen Builds, inklusive DWARF, für jeden Kern die eigene Datei. Hash oder Build-ID notieren.

Notizen

30 / 40

Ein zweiter Debug-Build kann andere Zeiten haben als der optimierte Lauf.

Folie

31 / 40

Die Magie der .elf-Datei

- Adressen, Typen, manchmal Zeilen
- mehrere Befehle je Zeile, Zeilen ohne Code
- der Decoder braucht dasselbe Image wie der Chip
- ein anderes ELF erklärt den Trace falsch, ohne dass das Tool „lügt“

Notizen

31 / 40

Fehlende Sync-Pakete: Abschnitt ungültig für Summen und Coverage.

Der Profiler und Trigger

- Trigger brauchen freie Komparatoren und eine Route
- `error_flag == 1` ist ein Beispiel, keine eingebaute Fähigkeit jeder Box
- „10 Sekunden alles“ hängt an Puffer und Rate
- Für die Ursache oft ein Ring mit Pre-Trigger, nicht erst ab dem Fehler starten

Notizen

32 / 40

Qualität vor der Auswertung: Sync, ELF, keine Overflows, bekannte Zeitbasis.

Folie

33 / 40

Das Problem der OS-Sichtbarkeit

`pxCurrentTCB` im lokalen M4-Kernel: die **ausgewählte** Task.

Während der ISR zeigt der Zeiger oft weiter auf sie, obwohl kein Taskcode läuft.

Der Wechsel liegt im Context Switch und ist nicht zwingend der erste Zyklus der neuen Task.

Ein wiederverwendeter TCB-Pointer ist keine ewige ID. Layout der eingesetzten Kernelversion prüfen. SMP-Strukturen sind hier nicht der M4-Kernel.

Notizen

33 / 40

Running-Profil ist nicht die Ready/Blocked-Historie.

Folie

34 / 40

OS-Awareness (Betriebssystem-Bewusstsein)

Zwei Stufen:

1. Running: aus `pxCurrentTCB` und dem Trace, ISR und PendSV daneben
2. Zustände Ready/Blocked: zusätzliche Instrumentierung, Aufwand kennzeichnen

Debug-Listen sind keine lückenlose Live-Historie.

ORTI/OSEK bleibt etwas anderes als FreeRTOS-Awareness. Queues können extra Registrierung brauchen.

Notizen

34 / 40

Frage: Was liefert der Zeiger während der ISR? Die zuletzt ausgewählte Task, nicht „die ISR ist diese Task“.

Wie winIDEA FreeRTOS visualisiert

Nur zusagen, was die installierte Hilfe zu **dieser** Version beschreibt.

Keine erfundenen Screenshots aus dem Netz als Laborfoto.

Beide Kernspuren gleichzeitig nur, wenn beide Quellen wirklich korreliert aufgenommen werden.

Sonst zwei Aufnahmen und keine Subtraktion der Zeitachsen.

Notizen

35 / 40

Unabhängige Zykluszähler nicht voneinander abziehen.

Folie

36 / 40

Der OSEK/ORTI-Standard (Automotive)

- ORTI beschreibt OSEK/AUTOSAR-OS-Objekte
- FreeRTOS-Awareness ist ein anderes Plugin und ein anderes Layout
- Beides nicht vermischen, auch wenn beide „Tasks“ zeichnen

Notizen

36 / 40

Für dieses Labor zählt der M4-FreeRTOS-Kern.

Folie

37 / 40

Laborvorbereitung: Tracing im Einsatz

Ohne Spezialhardware: GPIO um die Funktion oder CYCCNT desselben Kerns. 1000 Takte sind als Lehrsrechnung $2,5 \mu\text{s}$ am M7 (400 MHz) und $5 \mu\text{s}$ am M4 (200 MHz), keine Boardmessung. Den Zähler des anderen Kerns nicht subtrahieren. Versuch A bleibt die bekannte Variable; ein fehlendes Anfangspaket ist nicht der Wert 0.

Versuch B: non-cacheable Ping-Pong aus VL6. Druck 1, 3, 5. Die 500 ms liegen außerhalb der IPC-Messung.

Versuch C: separates Image, D-Cache an, Region nachweislich Write-Back. Hypothese, kein Drehbuch: M7-Store 1 kann da sein, M4 liest weiter 0. Kein garantierter Freeze. Timeout und Abbruch wie in VL6.

Versuch D: ein Job mit definierten Kanten. Min, Max, Anzahl, Auflösung, Bedingungen. Verluste machen die Summe ungültig.

Zeitquellen: M7 400 MHz nur wenn der Tree so steht, dann 1000 Takte = 2,5 μ s. M4 200 MHz: 1000 Takte = 5 μ s. Trace- und Capture-Zeit sind weitere Uhren. Ein Feld „400 MHz“ füllt nicht jede Achse.

Schritt 3–4: FreeRTOS Awareness & Profiler

Running-Profil der M4-Task: wiederkehrendes Delay ist Blocked/Ready/Running im Polling, kein eingefrorener Kern.

`osDelay(1)` ist ein Tick. Die Poll-Latenz ist nicht nur die Umschaltzeit.

Zustandshistorie nur mit der zusätzlichen Konfiguration und als instrumentierte Variante beschriftet.

PendSV auf einer eigenen Spur. Unter der ISR bleibt der Taskzeiger stehen, die CPU-Zeit gehört der ISR.

Schritt 5: Die Aufnahme & Analyse

Diagnose, je eine Prüfung:

Symptom	Prüfung
keine Verbindung	Versorgung, Reset-Reihenfolge, richtiger Adapter
nur ein Kern	ELF und Core-Zuordnung
keine Datenpakete	Komparator, Adresse, Breite, Startzeit
falsche Achse	Zeitquelle, nicht nur 400 MHz
Anfang fehlt	nicht als 0 werten
Verluste	Abschnitt verwerfen
falsche Tasks	Kernelversion, TCB-Layout
nur Running	State-Profiling fehlt
Cachefehler bleibt aus	CCR, MPU, Neustart, Debug-Cachewartung aus

Automatische Cachewartung und Live-Memory während Versuch C vermeiden oder als Störung protokollieren.

Zusammenfassung des Tages

- Messfrage, Quelle, Grenze
- 434 μ s Leitungszeit, 3 ms CPU, 6 ms Antwort, 1 ms Slack im Lehrjob
- DWT am CPU-Zugriff, nicht als SRAM-Monitor
- ETM rekonstruiert Pfade, solange das Bild vollständig ist
- $\max(C)$ ist kein WCET-Beweis
- Hardwareliste und gleichzeitige Dual-Core-Traces bleiben offen

Notizen

40 / 40

Quellen: Cortex-M7 TRM, CMSIS ITM/NVIC, UM2408, H755-Datenblatt und RM0399 der Revision, AN4839, iSYSTEM-Hilfen der vorhandenen winIDEA-Version, GCC $-g$, NASA MC/DC-Tutorial mit DO-178B-Kontext, FAA AC 20-115D. Keine Labor-Screenshots aus dem Netz.